

Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

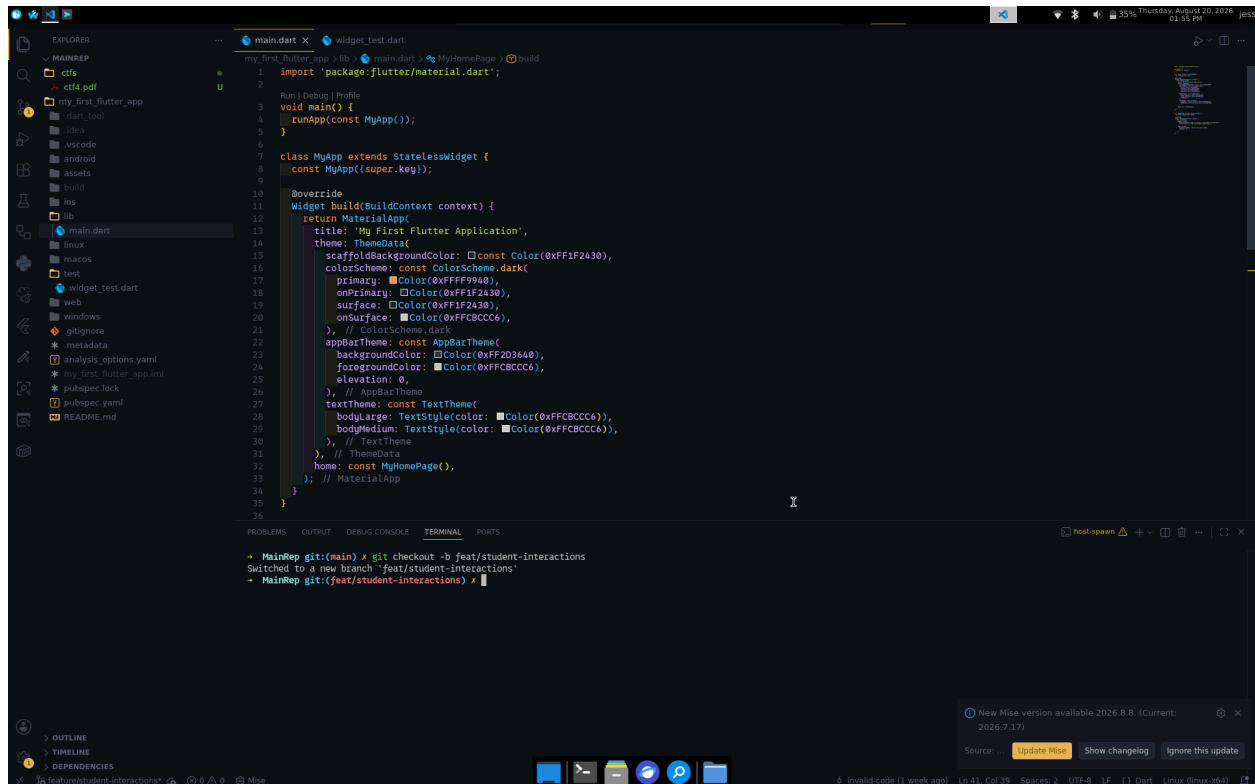
Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

- Terminal showing your current branch



🚩 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using `onPressed`.

Right now your buttons are only decorations.

Research how Flutter's `onPressed` works.

The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

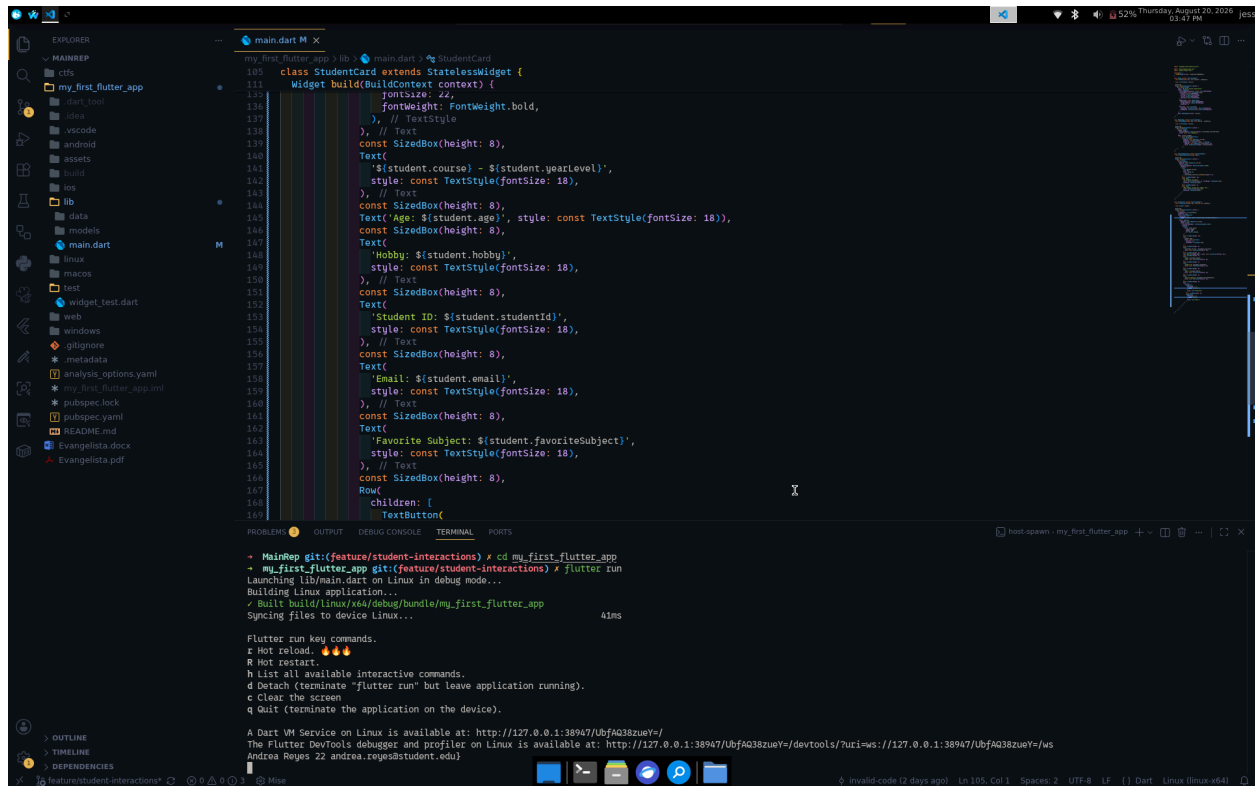
Screenshot:

- Student Card with buttons
- `onPressed` inside your code
- Button reacting or terminal message

```

111 class StudentCard extends StatelessWidget {
112   Widget build(BuildContext context) {
113     // Center
114     const SizedBox(height: 16),
115     Text(
116       student.name,
117       style: const TextStyle(
118         fontSize: 22,
119         fontWeight: FontWeight.bold,
120       ), // Text
121     ), // Text
122     const SizedBox(height: 8),
123     Text(
124       '${student.course} - ${student.yeastLevel}',
125       style: const TextStyle(fontSize: 18),
126     ), // Text
127     const SizedBox(height: 8),
128     Text('Age: ${student.age}', style: const TextStyle(fontSize: 18)),
129     const SizedBox(height: 8),
130     Text('Hobby: ${student.hobby}', style: const TextStyle(fontSize: 18)),
131     // Text
132     const SizedBox(height: 8),
133     Text('Student ID: ${student.studentId}', style: const TextStyle(fontSize: 18)),
134     // Text
135     const SizedBox(height: 8),
136     Text('Email: ${student.email}', style: const TextStyle(fontSize: 18)),
137     // Text
138     const SizedBox(height: 8),
139     Text('Favorite Subject: ${student.favoriteSubject}', style: const TextStyle(fontSize: 18)),
140     // Text
141     const SizedBox(height: 8),
142     Row(
143       children: [
144         TextButton(
145           onPressed: () {
146             print("favorite");
147           },
148           child: Text("Favorite"),
149         ), // TextButton
150         const SizedBox(width: 8),
151         TextButton(
152           onPressed: () {
153             print("edit");
154           },
155           child: Text("Edit"),
156         ), // TextButton
157       ], // Row
158     ),
159   }, // Row
160 },

```



🚩 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

Test

The user should be able to:

- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

Screenshot:

- **onTap** in your code
- Student Card being tapped

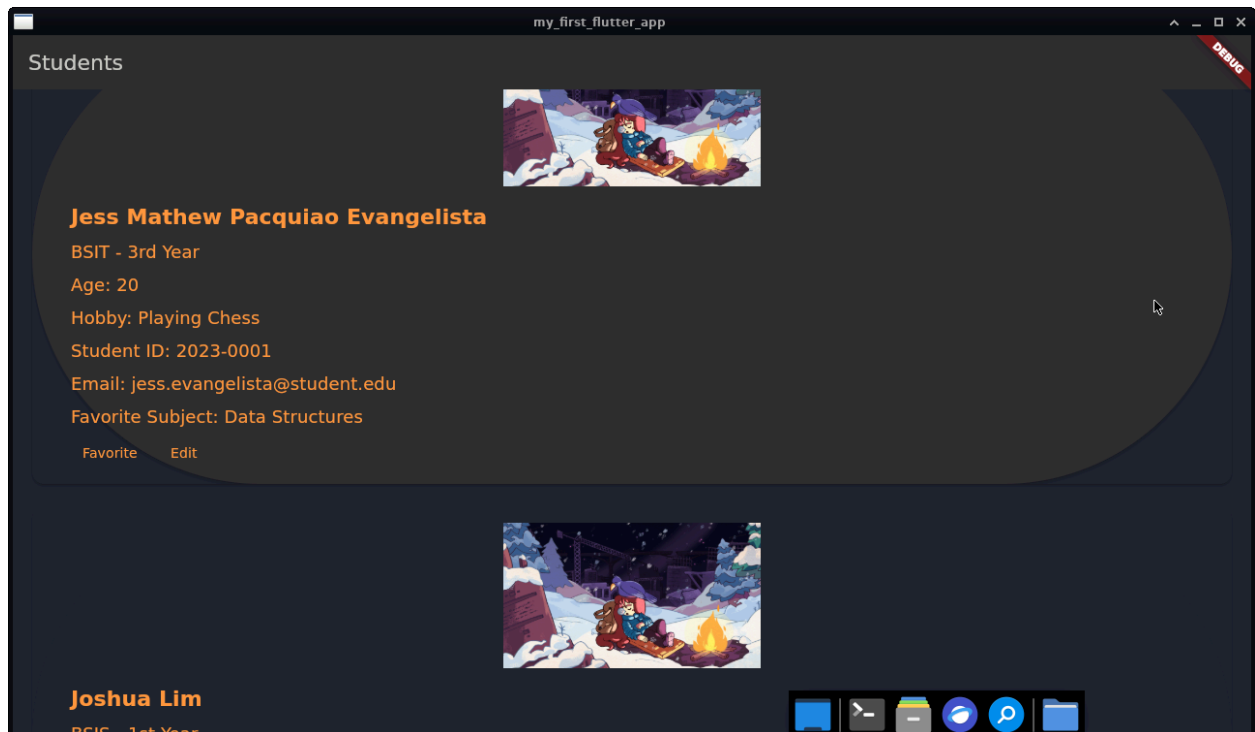
```

main.dart M
class EmptyStudentsView extends StatelessWidget {
  Widget build(BuildContext context) {
  }
}

class StudentCard extends StatelessWidget {
  const StudentCard(super.key, required this.student);
  final Student student;

  @override
  Widget build(BuildContext context) {
    return Card(
      clipBehavior: Clip.antiAlias,
      elevation: 2.0,
      child: ElevatedButton(
        onPressed: () {
          print('${student.name} ${student.age} ${student.email}');
        },
        child: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Center(
                child: Image.asset(
                  student.image,
                  height: 150,
                  fit: BoxFit.contain,
                ), // Image.asset
              ), // Center
              const SizedBox(height: 16),
              Text(
                student.name,
                style: const TextStyle(
                  fontSize: 22,
                  fontWeight: FontWeight.bold,
                ), // TextStyle
              ), // Text
              const SizedBox(height: 8),
              Text(
                '${student.course} - ${student.yearLevel}',
                style: const TextStyle(fontSize: 18),
              ), // Text
              const SizedBox(height: 8),
              Text('Age: ${student.age}', style: const TextStyle(fontSize: 18)),
              const SizedBox(height: 8),
              Text(
                'Hobby: ${student.hobby}',
                style: const TextStyle(fontSize: 18),
              ), // Text
              const SizedBox(height: 8),
              Text(
                'Student ID: ${student.studentId}',
                style: const TextStyle(fontSize: 18),
              ), // Text
              const SizedBox(height: 8),
            ],
          ),
        ),
      ),
    );
  }
}

```



🚩 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

The important part is understanding that `setState` tells Flutter to rebuild the UI.

Test

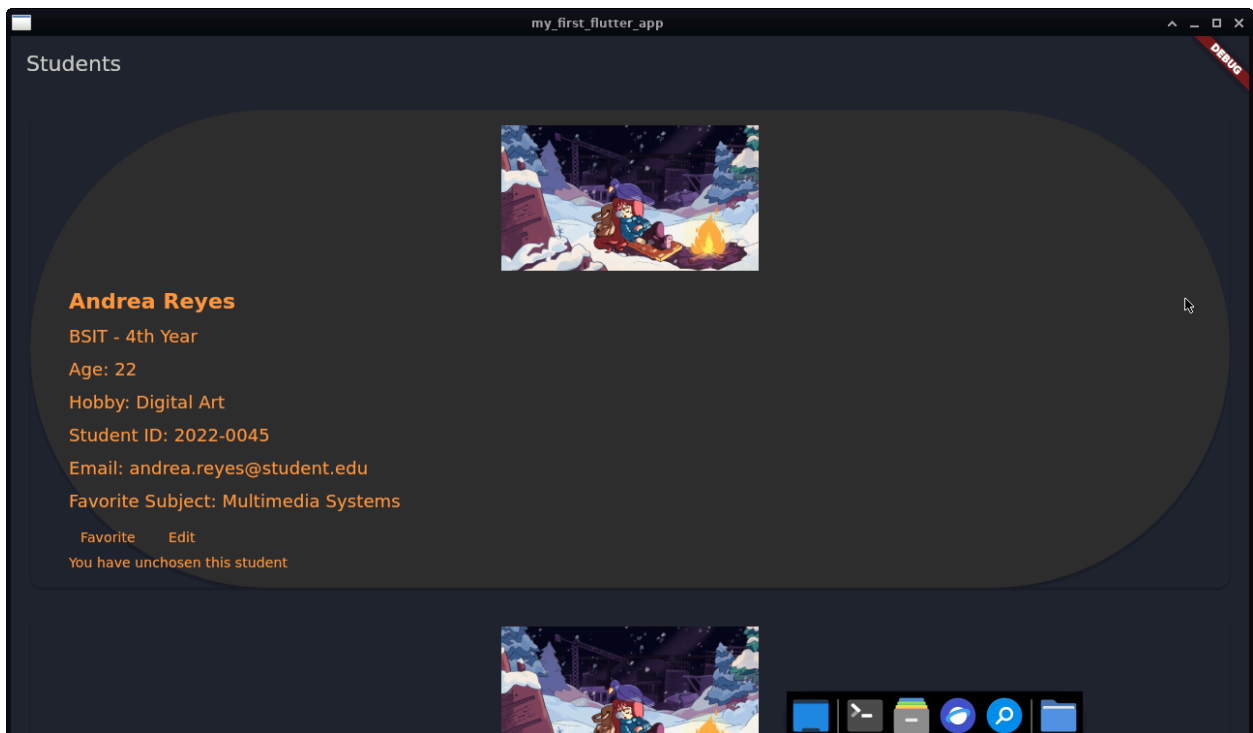
The screen should visibly change immediately after pressing the button.

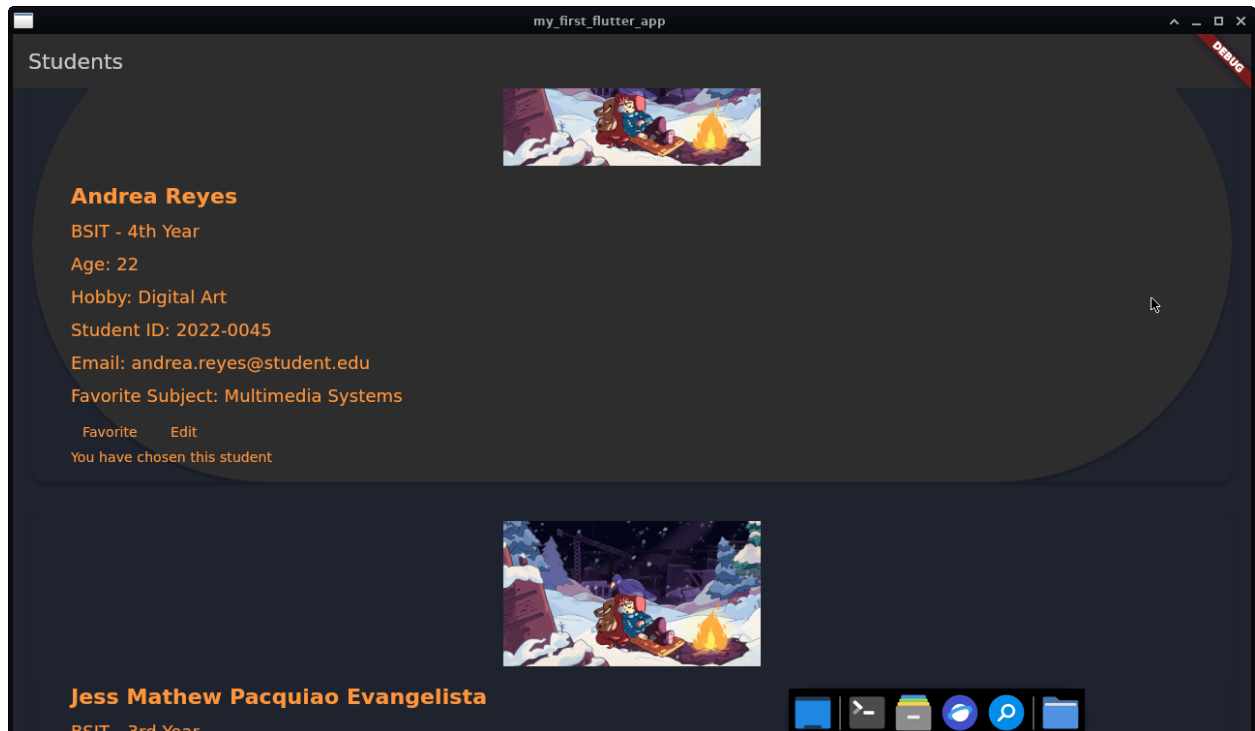
Screenshot:

- `setState` in your code
- Before and after interaction

```
EXPLORER
  my_first_flutter_app
    main.dart
    ...
  ...

main.dart
105 class ChoosableStudentCard extends StatefulWidget {
106   const ChoosableStudentCard({super.key, required this.student});
107   State<ChoosableStudentCard> createState() => _ChoosableStudentCardState();
108 }
109
110 class _ChoosableStudentCardState extends State<ChoosableStudentCard> {
111   bool isChosen = false;
112
113   @override
114   Widget build(BuildContext context) {
115     return Card(
116       clipBehavior: Clip.antiAlias,
117       elevation: 2.0,
118       child: ElevatedButton(
119         onPressed: () {
120           setState(() {
121             isChosen = !isChosen;
122           });
123         },
124         child: Padding(
125           padding: const EdgeInsets.all(16),
126           child: Column(
127             crossAxisAlignment: CrossAxisAlignment.start,
128             children: [
129               Center(
130                 child: Image.asset(
131                   widget.student.image,
132                   height: 150,
133                   fit: BoxFit.contain,
134                 ), // Image.asset
135               const SizedBox(height: 16),
136               Text(
137                 widget.student.name,
138                 style: const TextStyle(
139                   fontSize: 22,
140                   fontWeight: FontWeight.bold,
141                 ), // TextStyle
142               ), // Text
143               const SizedBox(height: 8),
144               Text(
145                 '${widget.student.course} - ${widget.student.yearLevel}',
146                 style: const TextStyle(fontSize: 18),
147               ), // Text
148               const SizedBox(height: 8),
149               Text(
150                 'Age: ${widget.student.age}',
151                 style: const TextStyle(fontSize: 18),
152               ), // Text
153               const SizedBox(height: 8),
154               Text(
155                 'Hobby: ${widget.student.hobby}',
156                 style: const TextStyle(fontSize: 18),
157               ), // Text
158             ],
159           ),
160         ),
161       ),
162     );
163   }
164 }
```





🚩 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

Each student's favorite status should work independently.

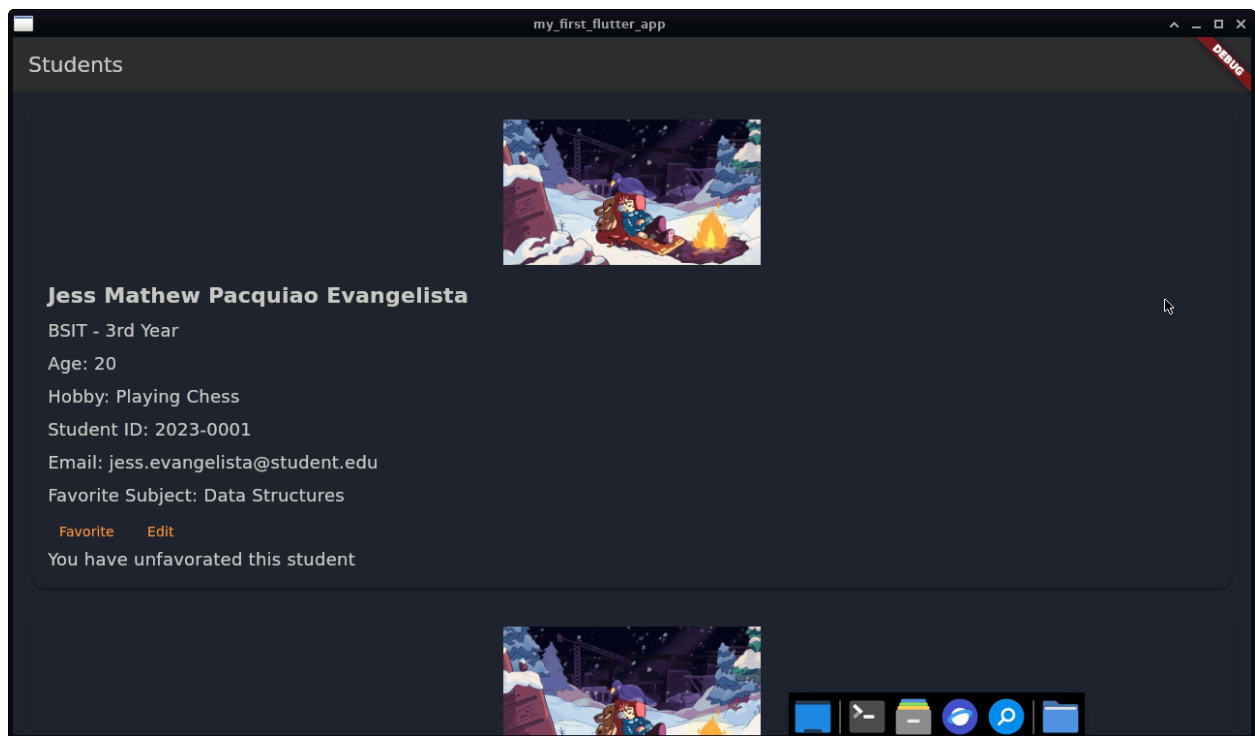
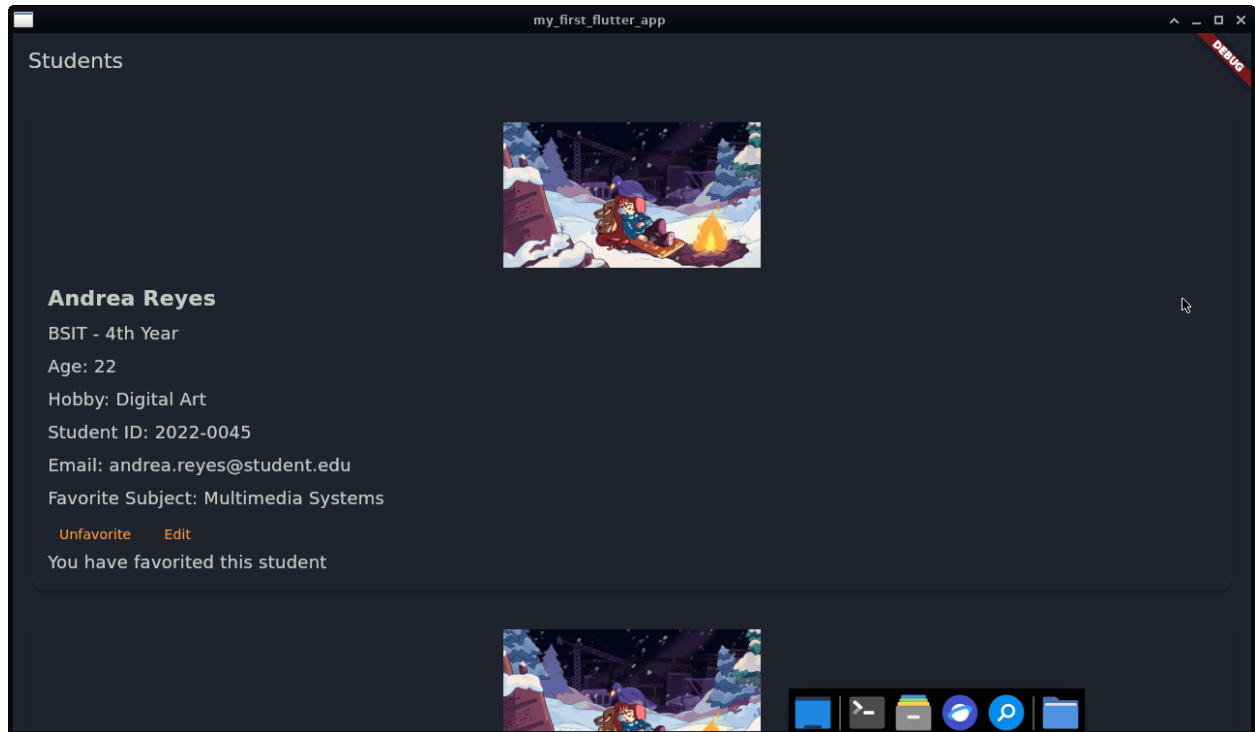
Test

- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

Both should keep their own state.

Screenshot:

- Favorited student
- Non-favorited student
- Favorite icon changing



Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart [List](#).

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

- Student A
- Student B
- Student C

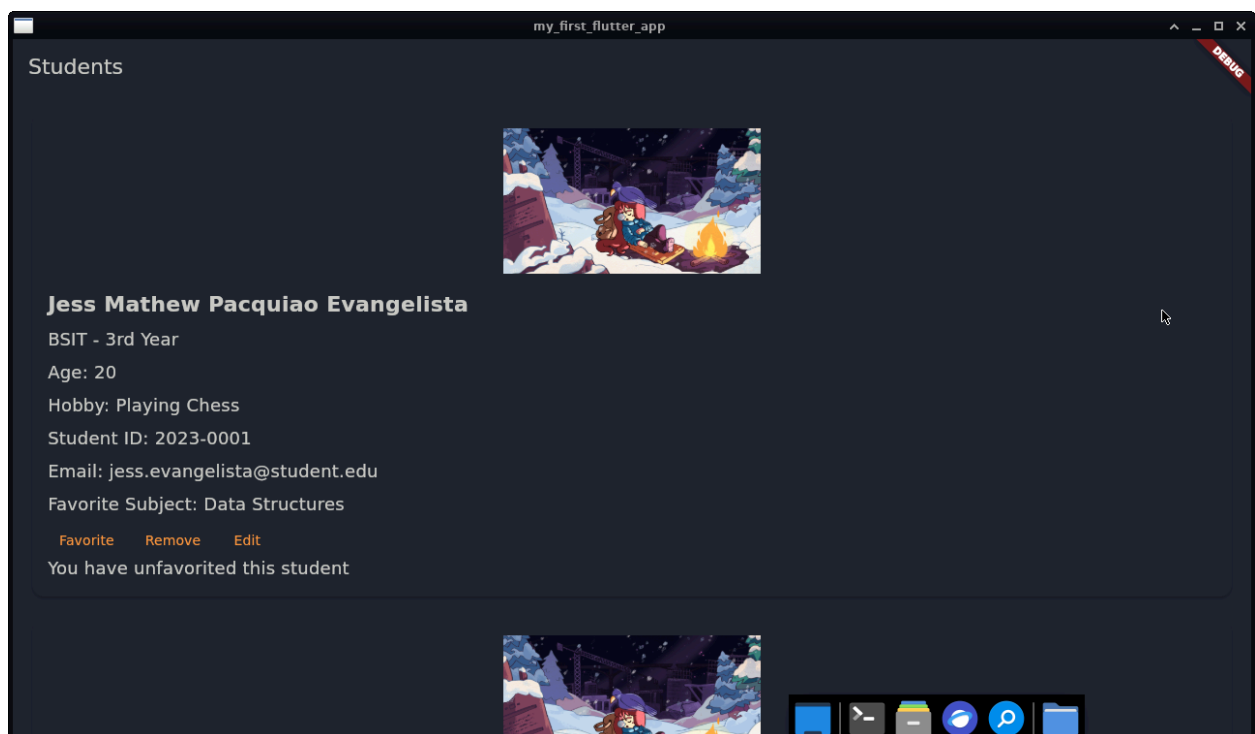
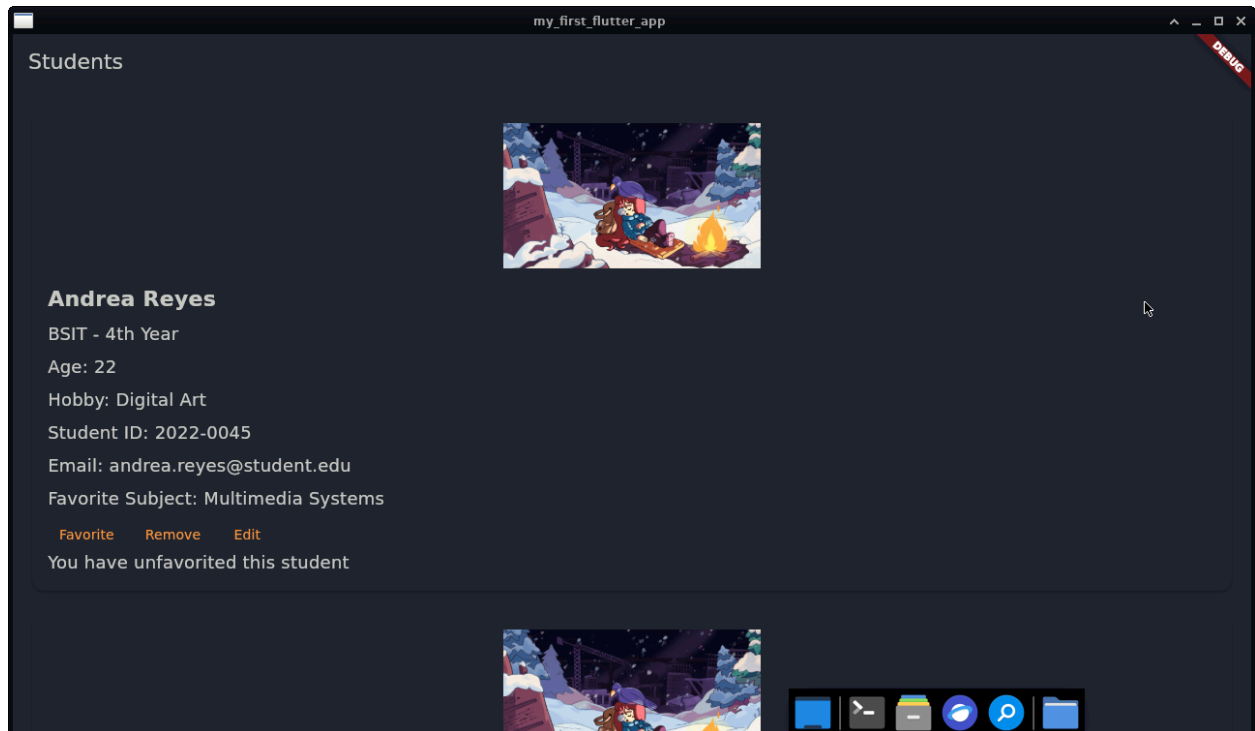
After deleting Student B:

- Student A
- Student C

No empty spaces should remain.

Screenshot:

- Student before deletion
- Student after deletion



🚩 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

- `AlertDialog`
- `SnackBar`
- Bottom Sheet

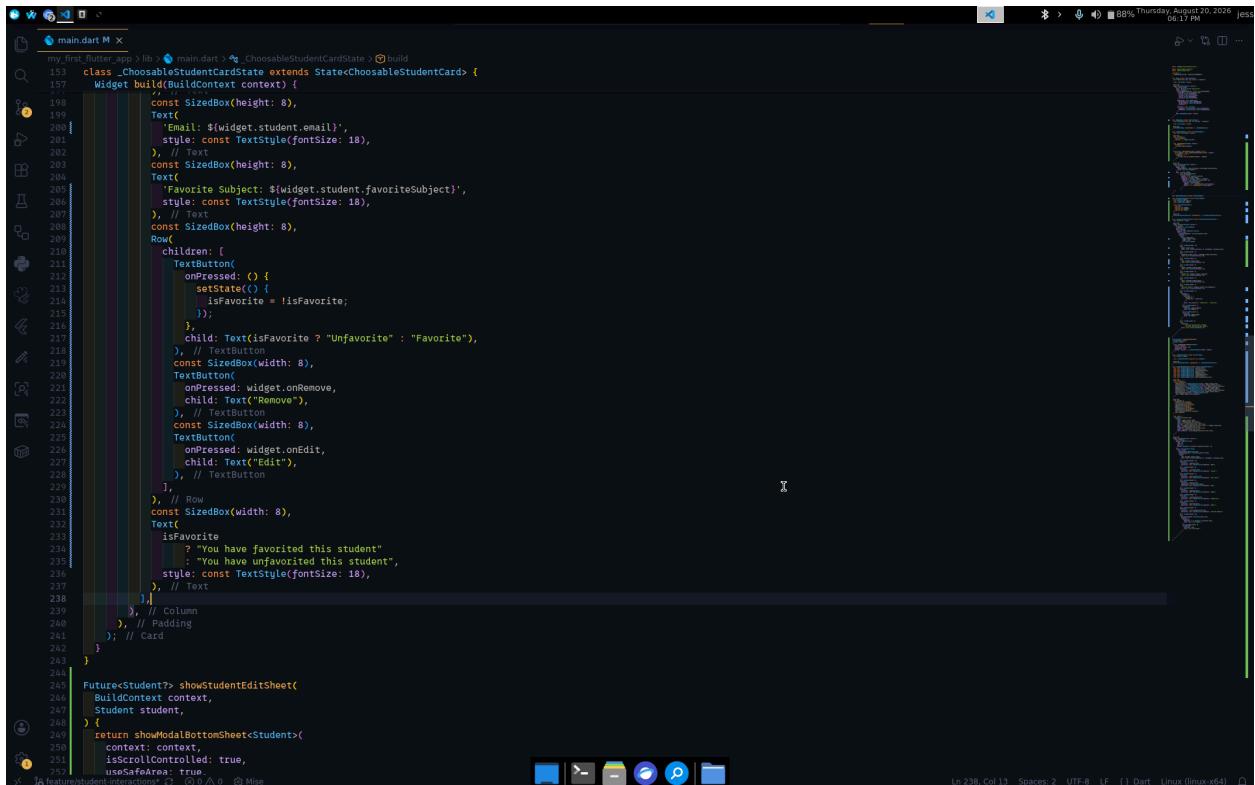
The goal is simply to launch an edit action, not build the complete editor yet.

Test

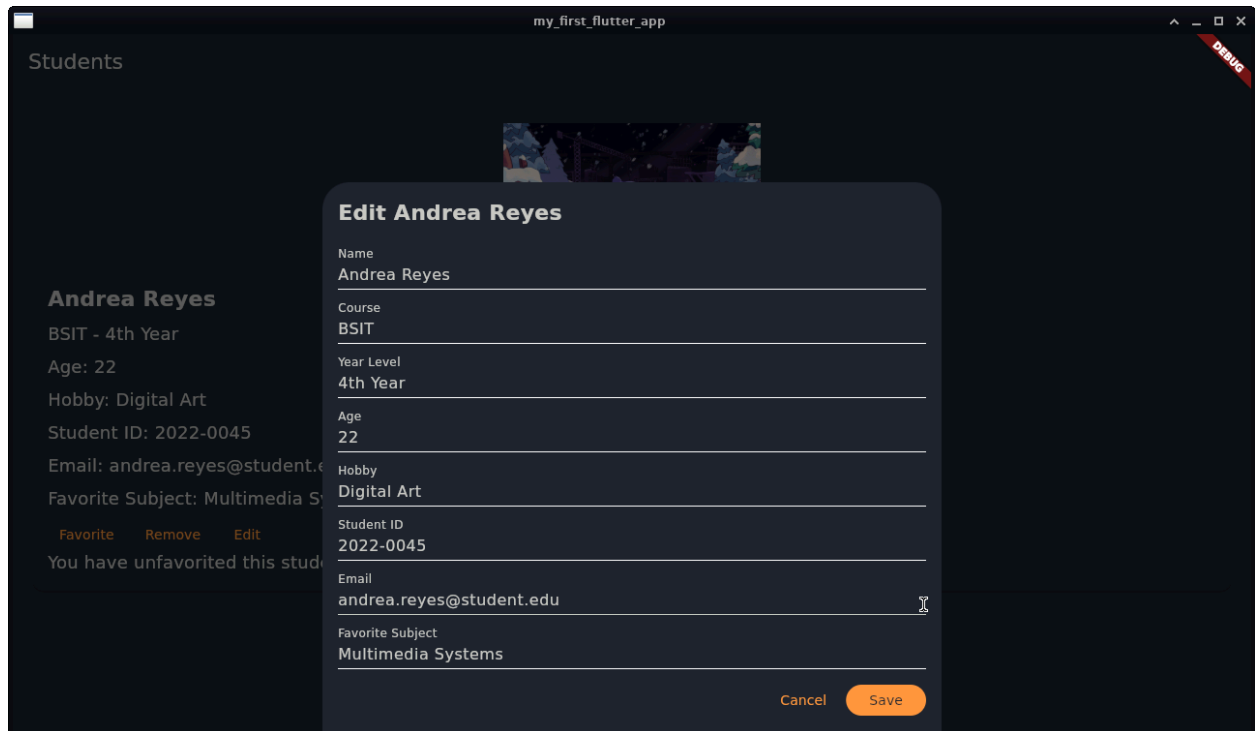
Pressing Edit should open your chosen interface.

Screenshot:

- Edit button
- Opened dialog, snackbar, or bottom sheet



```
main.dart M x
my_first_flutter_app > lib > main.dart > _ChoosableStudentCardState > build
153 class _ChoosableStudentCardState extends State<ChoosableStudentCard> {
157   Widget build(BuildContext context) {
158     const SizedBox(height: 8),
159     Text(
160       "Email: ${widget.student.email}",
161       style: const TextStyle(fontSize: 18),
162     ), // Text
163     const SizedBox(height: 8),
164     Text(
165       "Favorite Subject: ${widget.student.favoriteSubject}",
166       style: const TextStyle(fontSize: 18),
167     ), // Text
168     const SizedBox(height: 8),
169     Row(
170       children: [
171         TextButton(
172           onPressed: () {
173             setState(() {
174               isFavorite = !isFavorite;
175             });
176           },
177           child: Text(isFavorite ? "Unfavorite" : "Favorite"),
178         ), // TextButton
179         const SizedBox(width: 8),
180         TextButton(
181           onPressed: widget.onRemove,
182           child: Text("Remove"),
183         ), // TextButton
184         const SizedBox(width: 8),
185         TextButton(
186           onPressed: widget.onEdit,
187           child: Text("Edit"),
188         ), // TextButton
189       ],
190     ), // Row
191     const SizedBox(width: 8),
192     Text(
193       isFavorite
194         ? "You have favorited this student"
195         : "You have unfavorited this student",
196       style: const TextStyle(fontSize: 18),
197     ), // Text
198   ], // Column
199 ), // Padding
200 ], // Card
201 }
202
203 Future<Student?> showStudentEditSheet(
204   BuildContext context,
205   Student student,
206 ) {
207   return showModalBottomSheet<Student>(<
208     context: context,
209     isScrollControlled: true,
210     useSafeArea: true
211   );
212 }
```



🚩 Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.

Your application should continue functioning correctly without affecting unrelated students.

This is your final interaction challenge.

Verify that every student behaves independently.

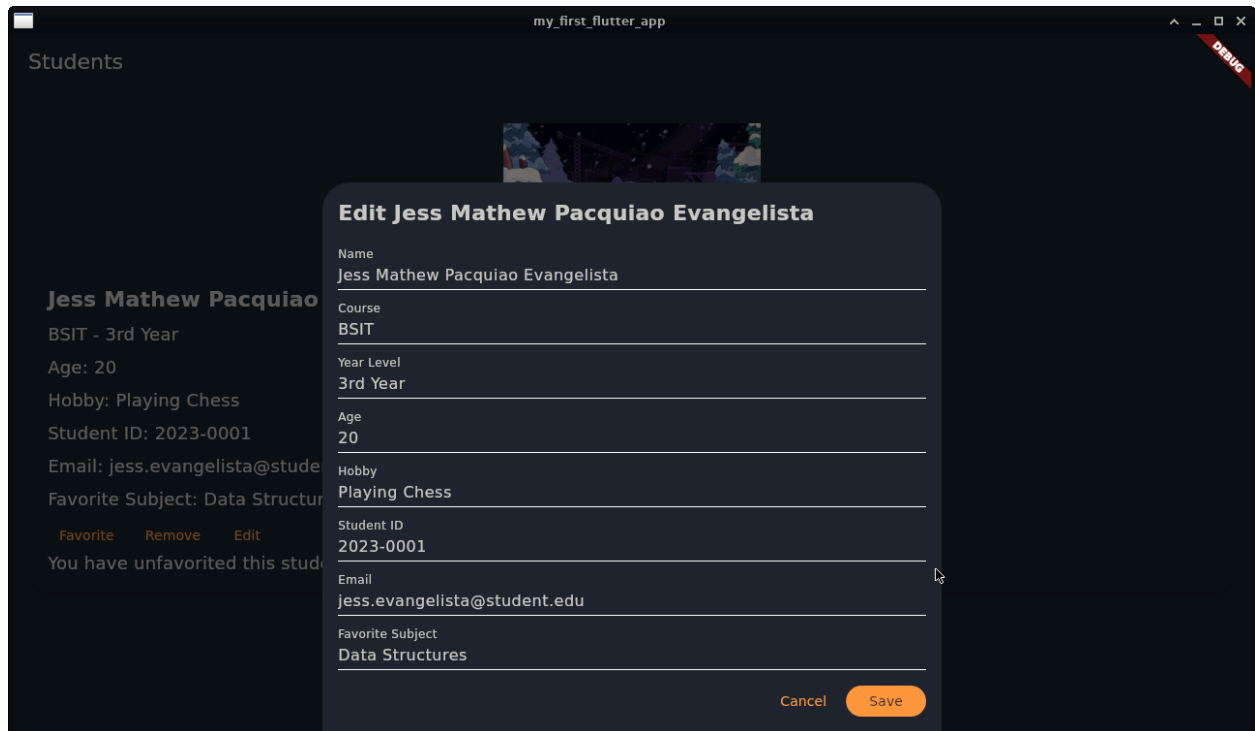
Example scenario:

Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:

- Multiple interactions working together
- Application still functioning normally



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

`feature/student-interactions`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

